

Automatic Visual Bug Detection — Project Summary

Scope, Methods, Results, and Next Steps

Prepared for: Internal Research • Author: Felix Morosov • Date: Oct 28, 2025

1. Project Overview

Goal: Build a data-driven system that detects visual rendering bugs in gameplay video. We focused on five canonical bug types and built an end-to-end pipeline: data collection, clip mining, a temporal deep model, and evaluation with strong validation performance.

2. Target Bug Types

- z-clipping
- corrupted_texture
- geometry_corruption
- z-fighting
- boundary_hole

3. What Has Been Done

- Background research and narrowed scope to five bug types.
- Implemented bug injection & logging scripts in three Unity games (Mario Kart 8-style clone, Five Nights at Freddy's-style clone, FPS Microgame).
- Built a clip miner that fuses gameplay videos with bug logging JSONL to produce positive/negative clips at the desired FPS and duration.
- Ran a user study: recorded gameplay from 11 participants across all three games, capping simultaneous bugs at 3.
- Constructed a large-scale dataset (~78k clips; ~1.2M frames).
- Developed an AI model: ResNet-18 (frame encoder) → BiLSTM + Temporal Attention → three heads (any-bug, count, bug-type).

4. Dataset Summary

Games	3 (Unity): MK8-style clone, FNaF-style clone, FPS Microgame
Participants	11
Clips	≈ 78,000
Frames	≈ 1.2 million
Max simultaneous bugs per clip	3

5. Model Architecture

Backbone: ResNet-18 (ImageNet-initialized), shared over frames.

Temporal encoder: 2-layer BiLSTM (hidden=256, bidirectional) + Temporal Attention pooling.

Heads: Multi-label bug types (5, sigmoid), Count (4 buckets), Any-bug (sigmoid).

Training: AdamW, cosine LR with warmup; AMP; EMA; ASL for types; focal CE for count; BCE for any; consistency loss between Any and Types; negative down-weighting on clean clips; count-weighted sampler.

6. Results (Validation)

Gated Types F1: 92% (0.92)

Any-bug F1: ~0.77

Count accuracy: ~0.86

Notes: Gating + tuned thresholds substantially boost per-class performance; recall-heavy any-bug threshold avoids misses, while class thresholds recover precision.

7. Future Work (for Nahian)

- Experiment with alternative backbones/architectures (3D CNNs, TimeSformer, ConvNext, SlowFast).
- Zero-shot generalization: evaluate on a new, unseen game.
- Expand taxonomy: add UI/menu bugs and seed examples in the dataset.
- Author more games/scenes to diversify visual contexts.
- Include 4- and 5-bug clips to stress-test multi-label capacity.
- Ablation studies: gating, attention, losses, sampling, thresholds, EMA, augmentation.
- Vary FPS and clip length to understand temporal resolution vs. compute tradeoffs.
- Visualization: class activation maps / attention rollout; clip-level bug timelines.
- End-to-end pipeline: raw gameplay → clipper → model inference → bugged clip retrieval & timestamps.

8. Takeaway

We now have a scalable dataset, a reliable clip-mining process, and a high-performing temporal model. The system achieves strong validation performance ($\approx 92\%$ gated F1) across five bug types and is positioned for generalization studies and pipeline productization.